

DeepSeek-R1本地部署 (Ubuntu+vLLM)



Stone-Wave
河北工业大学 计算机技术硕士

关注他

3 人赞同了该文章 >

一、部署方式

目前主流的大模型推理框架主要有以下四种：**SGLang**、**Ollama+**、**VLLM+**和 **LLaMA.cpp**，下面是四种架构的基本信息以及对比差异。

推理框架	性能表现	易用性	适用场景	硬件需求	模型支持	部署方式	系统支持
SGLang	零开销批处理提升 1.1 倍吞吐量，缓存感知负载均衡提升 1.9 倍，结构化输出提速 10 倍	需一定技术基础，但提供完整 API 和示例	企业级推理服务、高并发场景、需要结构化输出的应用	高端 GPU 推荐 A100/H100，支持多 GPU 部署	全面支持主流大模型，特别优化 DeepSeek 等模型	Docker、Python 源码	Linux
Ollama	继承 llama.cpp 的高效推理能力，提供便捷的模型管理沙盒	小白友好，提供图形界面安装程序一键运行命令，支持 REST API	个人开发者项目验证、学生辅助学习、日常问答、创意写作等个人轻量级应用	与 llama.cpp 相同，但提供更简便的资源管理	模型库丰富，涵盖 1700 多款，支持一键下载安装	独立应用程序安装包	Windows、macOS、Linux
vLLM	高效性能，通 Paged-	需一定技术基础，特	大模型开发与	CPU/GPU，支持多机	广泛的模型支持	Docker、	Linux、macOS

	Attention 技术和 Continuous Batching 技术提高效率，吞吐量相较于原生 HF Transformers 可提升高达 24 倍，支持多种量化技术进一步压缩模型显存占用	别是对参数的配置	商业推理服务	多卡 GPU 部署		Python 源码	
LLaMA.cpp	极高性能，CPU/GPU 协同计算，甚至无 GPU 也可以推理，是没有显卡玩家的最大福音	相对复杂	商业应用与研究	CPU/GPU	专属 LLaMA 模型	本地部署、分布式部署	Linux、Windows

二、DeepSeek-R1系列模型选择

DeepSeek-R1系列模型在Huggingface+上共计开源了8种，按照参数量排序依次为：

DeepSeek-R1 671B

DeepSeek-R1-Zero 671B
DeepSeek-R1-Distill-Llama-70B
DeepSeek-R1-Distill-Qwen-32B
DeepSeek-R1-Distill-Qwen-14B
DeepSeek-R1-Distill-Llama-8B
DeepSeek-R1-Distill-Qwen-7B
DeepSeek-R1-Distill-Qwen-1.5B

除了DeepSeek-R1和DeepSeek-R1-Zero两个核心系列，其他不同参数规模的模型是以DeepSeek-R1作为教师模型使用不同的学生模型例如Qwen或者llama蒸馏而来，不同参数量要求的硬件门槛也各不相同。

模型参数	最低GPU配置	最低CPU配置	建议内存	建议硬盘空间
R1-满血版	A/H100(80G)x18	Xeon 8核	192GB	2TB固态
R1-70B	RTX 4090(24GB)x2	i9-13900K	64GB	1TB固态
R1-32B	RTX 4090(24GB)	i7-13700K	64GB	1TB固态
R1-14B	RTX 4060S(16GB)	Ryzen 7	32GB	500G固态

三、vLLM部署

vLLM是本次部署模型推理采用的框架，选择vLLM的原因

vllm对并发需求支持比Ollama要好。
vllm可以把模型封装成openAI格式的API接口，除了在本地使用之外，还可以通过暴露端口到公网IP的方式对外提供API服务。

1、下载vllm

vllm代码地址：

GitHub - vllm-project/vllm: A high-throughput and memory-efficient inference and serving engine f...
🔗github.com/vllm-project/vllm

使用git把代码拉到本地使用git把代码拉到本地

```
git clone https://github.com/vllm-project/vllm.git
```

2、下载模型文件

有两种途径：**Huggingface**和**modelscope**

Huggingface

```
https://huggingface.co/models?
sort=trending&search=deepseek-r1
🔗huggingface.co/models?sort=trending&search=deeps...
```

多种下载方式

- 1、通过网页端手动一个一个下载模型文件。
- 2、使用Huggingface-cli命令一次性拉取整个模型文件夹到本地路径。使用Huggingface命令行下载的话需要简单进行一些设置。

```
pip install -U huggingface_hub
```

设置Huggingface的镜像站环境变量

```
export HF_ENDPOINT=https://hf-mirror.com
```

使用Huggingface-cli命令进行下载

```
huggingface-cli download Huggingface项目的链接地址 --local-dir 保存的模型路径
eg:
70B huggingface-cli download deepseek-ai/DeepSeek-R1-Distill-Llama-70B --local-dir model
32B huggingface-cli download deepseek-ai/DeepSeek-R1-Distill-Qwen-32B --local-dir model
AWQ: (Activation-Aware Weight Quantization, 激活感知权重量化)
70B huggingface-cli download Valdemardi/DeepSeek-R1-Distill-Llama-70B-AWQ --local-dir model
32B huggingface-cli download Valdemardi/DeepSeek-R1-Distill-Qwen-32B-AWQ --local-dir model
GGUF量化(GPT-Generated Unified Format, GPT 生成的统一格式)
70B huggingface-cli download unsloth/DeepSeek-R1-Distill-Qwen-32B-GGUF --local-dir model
32B huggingface-cli download unsloth/DeepSeek-R1-Distill-Qwen-32B-GGUF --local-dir model
```

modelscope



ModelScope - 模型列表页

modelscope.cn/models?name=deepseek-...

多种下载方式

1、modelscope (2选1)

首先安装modelscope

```
pip install modelscope
```

1) modelscope命令下载

`modelscope download 链接地址 --local-dir 保存的模型路径`

eg:

```
70B modelscope download --model deepseek-ai/DeepSeek-R1-Distill-Llama-70B --local_dir mc
```

```
32B modelscope download --model deepseek-ai/DeepSeek-R1-Distill-Qwen-32B --local_dir moc  
AWQ
```

```
70B modelscope download --model Valdemardi/DeepSeek-R1-Distill-Llama-70B-AWQ --local_dir
```

```
32B modelscope download --model Valdemardi/DeepSeek-R1-Distill-Qwen-32B-AWQ --local_dir  
GGUF
```

```
70B modelscope download --model unsloth/DeepSeek-R1-Distill-Qwen-32B-GGUF --local_dir mc
```

```
32B modelscope download --model unsloth/DeepSeek-R1-Distill-Qwen-32B-GGUF --local_dir mc
```

2) SDK下载

#模型下载

```
from modelscope import snapshot_download
```

```
model_dir = snapshot_download('deepseek-ai/DeepSeek-R1-Distill-Llama-70B') # 70B
```

```
model_dir = snapshot_download('deepseek-ai/DeepSeek-R1-Distill-Qwen-32B') # 32B
```

2、Git下载

请确保 lfs 已经被正确安装

```
git lfs install
```

git命令

```
git clone https://www.modelscope.cn/deepseek-ai/DeepSeek-R1-Distill-Llama-70B.git
git clone https://www.modelscope.cn/deepseek-ai/DeepSeek-R1-Distill-Qwen-32B.git
```

3、docker使用

docker安装

安装依赖

```
sudo apt-get install ca-certificates curl gnupg lsb-release
sudo apt-get -y install apt-transport-https ca-certificates curl software-properties-common
```

安装GPG证书

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

设置稳定版仓库

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list >/dev/null
```

安装 (以下2选1)

最新版本

```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

特定版本

```
apt-cache madison docker-ce
```

```
(base) root@DeepSeek:~# apt-cache madison docker-ce
docker-ce | 5:28.0.4-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:28.0.3-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:28.0.2-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:28.0.1-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:28.0.0-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.5.1-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.5.0-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.4.1-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.4.0-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.3.1-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.3.0-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.2.1-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.2.0-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.1.2-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.1.1-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.1.0-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.0.3-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.0.2-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:27.0.1-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:26.1.4-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:26.1.3-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:26.1.2-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:26.1.1-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:26.1.0-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:26.0.2-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:26.0.1-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
docker-ce | 5:26.0.0-1~ubuntu.24.04~noble | https://mirrors.aliyun.com/docker-ce/linux/ubuntu noble/stable amd64 Packages
```

```
sudo apt-get install docker-ce=<VERSION_STRING> docker-ce-cli=<VERSION_STRING> containerd.io=<VERSION_STRING>
```

VERSION_STRING版本字符串表示安装特定版本，例如5:28.0.4-1~ubuntu.24.04~noble

安装NVIDIA Container Toolkit

使用中国科学技术大学（USTC）镜像源安装 NVIDIA Container Toolkit

1.下载 USTC 的 GPG 密钥:

```
curl -fsSL https://mirrors.ustc.edu.cn/libnvidia-container/gpgkey | sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit.gpg
```

2.配置 USTC 的 APT 源:

```
curl -s -L https://mirrors.ustc.edu.cn/libnvidia-container/stable/deb/nvidia-container-toolkit.deb | \
sed 's#deb https://nvidia.github.io#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit.gpg] https://mirrors.ustc.edu.cn/libnvidia-container/stable/deb/' | \
sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

3.更新 APT 包列表:

```
sudo apt-get update
```

4.安装 NVIDIA Container Toolkit:


```
sudo apt-get install -y nvidia-container-toolkit
```

5.验证安装:

```
nvidia-container-cli --version
```

安装 nvidia-docker2 包

```
sudo apt-get install -y nvidia-docker2
```

安装docker-compose

```
sudo apt-get install docker-compose
```

重启Docker daemon 来完成设定默认运行时后的安装

```
sudo systemctl restart docker
```

配置docker参数

创建一个docker启动的参数文件，文件名命名为docker-compose.yml

```
services:
  vllm:
    container_name: vllm
    restart: no
    image: vllm/vllm-openai
    ipc: host
    environment:
      TZ: Asia/Shanghai
      - CUDA_VISIBLE_DEVICES=0,1 # 指定使用 GPU 0 和 1
    volumes:
      - ./models:/models
    command: [
      "--model", "/models/DeepSeek-R1-32B-AWQ",
      "--served-model-name", "DeepSeek-R1-32B-AWQ",
      "--max-model-len", "18432",
      "--max-num-batched-tokens", "512",
      "--max-num-seqs", "1",
      "--quantization", "awq_marlin",
```



```
    "--host", "0.0.0.0",
    "--port", "8000",
    "--enforce-eager",
    "--gpu-memory-utilization", "0.95",
    "--enable-chunked-prefill",
    "--api-key", "your_api_key"
]
ports:
  - 8003:8000
deploy:
  resources:
    reservations:
      devices:
        - driver: nvidia
          count: all
          capabilities: [gpu]
```

各配置参数含义:

```
model: "./models/DeepSeek-R1-32B-AWQ" # 替换为你实际的模型路径
tensor_parallel_size: 2 # 使用双卡 4090
trust_remote_code: true
max_model_len: 18432 # 最大模型长度
max-num-batched-tokens: 512 # 设置批量处理的token数为512，平衡处理速度和显存占用。
gpu_memory_utilization: 0.95 # GPU 内存利用率
quantization: "awq_marlin" # 指定量化方式为激活感知权重 (AWQ)
enable-chunked-prefill: true # 启用分块预填充，减少显存碎片。
enforce-eager: true # 启用 eager 模式，加快推理速度。
port: 8000 #容器使用8000端口号
```

启动

```
docker compose up -d
```

查看日志

```
docker compose logs -f
```

```
Loading safetensors checkpoint shards: 0% Completed | 0/4 [00:00<?, ?it/s]
Loading safetensors checkpoint shards: 25% Completed | 1/4 [00:00<00:01, 1.70it/s]
Loading safetensors checkpoint shards: 50% Completed | 2/4 [00:01<00:01, 1.75it/s]
Loading safetensors checkpoint shards: 75% Completed | 3/4 [00:01<00:00, 1.86it/s]
Loading safetensors checkpoint shards: 100% Completed | 4/4 [00:02<00:00, 1.81it/s]
Loading safetensors checkpoint shards: 100% Completed | 4/4 [00:02<00:00, 1.80it/s]
vllm |
vllm | INFO 04-14 17:25:01 [loader.py:447] Loading weights took 2.27 seconds
vllm | INFO 04-14 17:25:02 [gpu_model_runner.py:1273] Model loading took 18.1693 GiB and 3.271710 seconds
vllm | INFO 04-14 17:25:03 [kv_cache_utils.py:578] GPU KV cache size: 107,408 tokens
vllm | INFO 04-14 17:25:03 [kv_cache_utils.py:581] Maximum concurrency for 18,432 tokens per request: 5.83x
vllm | INFO 04-14 17:25:03 [core.py:162] init engine (profile, create kv cache, warmup model) took 0.55 seconds
vllm | INFO 04-14 17:25:03 [api_server.py:1081] Starting vLLM API server on http://0.0.0.0:8003
vllm | INFO 04-14 17:25:03 [launcher.py:26] Available routes are:
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /openapi.json, Methods: HEAD, GET
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /docs, Methods: HEAD, GET
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /docs/oauth2-redirect, Methods: HEAD, GET
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /redoc, Methods: HEAD, GET
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /health, Methods: GET
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /load, Methods: GET
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /ping, Methods: GET, POST
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /tokenize, Methods: POST
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /detokenize, Methods: POST
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /v1/models, Methods: GET
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /version, Methods: GET
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /v1/chat/completions, Methods: POST
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /v1/completions, Methods: POST
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /v1/embeddings, Methods: POST
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /pooling, Methods: POST
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /score, Methods: POST
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /v1/score, Methods: POST
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /v1/audio/transcriptions, Methods: POST
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /rerank, Methods: POST
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /v1/rerank, Methods: POST
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /v2/rerank, Methods: POST
vllm | INFO 04-14 17:25:03 [launcher.py:34] Route: /invocations, Methods: POST
vllm | INFO: Started server process [1]
vllm | INFO: Waiting for application startup.
vllm | INFO: Application startup complete.
```

知乎 @Stone-Wave

当出现此信息时，说明模型已经加载并成功启动。

四、模型测试

使用图形显示框架openwebUI+

<https://github.com/open-webui/open-webui>

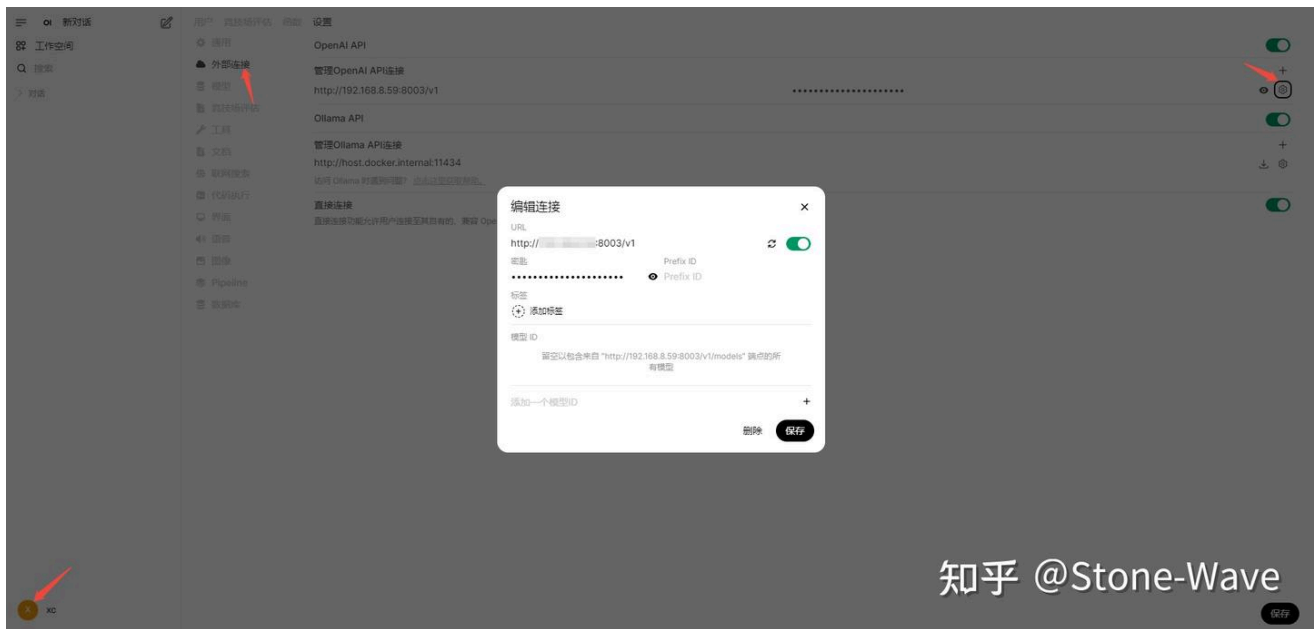
github.com/open-webui/open-webui

在终端中输入如下docker命令

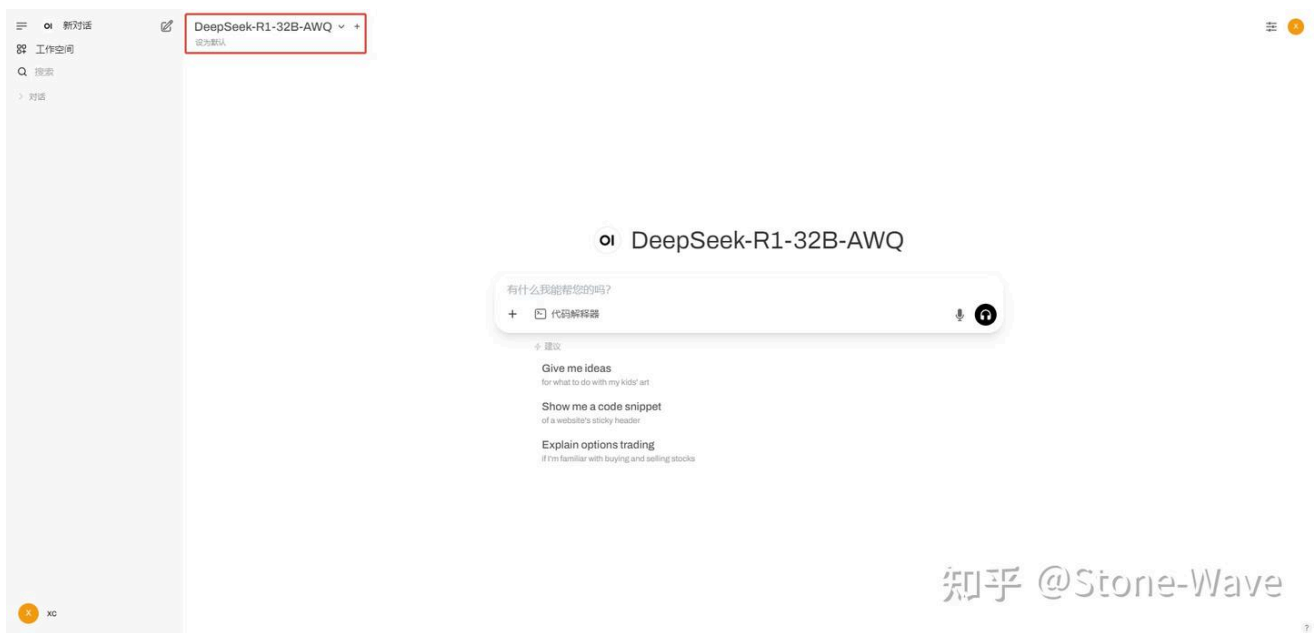
```
docker run -d -p 3000:8080 --add-host=host.docker.internal:host-gateway -v /data/open-v
```

可以在局域网内打开[http://\[Open WebUI服务器IP\]:3000/](http://[Open WebUI服务器IP]:3000/)

注册好账号后，点击左下角（或右上角）用户图标，点击设置->管理员设置->外部连接，配置“管理OpenAI API连接”，url写入[http://\[vLLM服务器IP\]:8003/v1](http://[vLLM服务器IP]:8003/v1)，密钥写入vLLM中docker-compose.yml设置的api-key



设置好后，可以点击刷新按钮（验证是否连通），保存后对话块会出现对应的模型。



OK到这里说明已经成功完成了将deepseek- R1模型部署为openAI格式的API接口，可以提供给局域网内的用户请求访问。